# Laboratorio 10 – Requisitos del software para la arquitectura cliente-servidor (versión Eclipse)

Oct 22, 2026 · @Alexis Flores Franco

Cada equipo entrega al final de la semana un SRS simplificado de su sistema cliente-servidor: requisitos funcionales y no funcionales medibles, framework justificado y tabla de endpoints. En esta versión, el proyecto de ejemplo se abre y se ejecuta en **Eclipse IDE**.

## Datos generales

| Campo | Detalle |
|---|---|
| Asignatura | Programación V (PROG 309) · Código 28186 |
| Eje temático | 3 · Patrón Cliente/Servidor |
| Semana | 10 · 19 al 23 de octubre de 2026 |
| Sesiones de laboratorio | Jueves 22 y viernes 23, 7:00–8:35 |
| Docente | Prof. Alexis Flores Franco |
| Modalidad | Equipos de 3 a 4 estudiantes (los mismos de la semana 9) |
| Material de apoyo | Proyecto ReservasLabAPI (Java 8, sin librerías externas), Eclipse IDE for Java Developers, presentación de la semana 10, Postman |
| Entrega | Plataforma institucional, domingo 25 de octubre de 2026, 11:59 p. m. |

## Objetivos

Al terminar el laboratorio, el estudiante podrá:

1. Redactar requisitos funcionales con identificador, actor, entradas, salidas y criterio de aceptación.
2. Formular requisitos no funcionales medibles de disponibilidad, latencia, seguridad y concurrencia.

3. Seleccionar y justificar el framework para implementar el servidor.
4. Documentar los endpoints que expondrá el servidor, con métodos, rutas y códigos HTTP.
5. Relacionar cada requisito con una decisión de diseño visible en el código.

## Marco teórico breve

Un requisito funcional (RF) describe QUÉ hace el sistema; en una arquitectura cliente-servidor suele convertirse en un endpoint. Un requisito no funcional (RNF) describe CÓMO de bien lo hace y se verifica con una métrica.

| Atributo (ISO/IEC 25010) | Por qué es crítico en cliente-servidor | Ejemplo medible |
|---|---|---|
| Disponibilidad | Si el servidor cae, caen todos los clientes | Servicio disponible el 99,5 % del mes |
| Rendimiento / latencia | Cada operación cruza la red | 95 % de solicitudes en ⩽ 500 ms con 20 clientes |
| Seguridad | La API está expuesta en la red | Escritura solo con X-API-KEY válida; sin ella → 401 |
| Concurrencia / escalabilidad | Muchos clientes llaman a la vez | 10 solicitudes atendidas en paralelo sin perder datos |
| Interoperabilidad | Clientes web, móvil y consola distintos | JSON UTF-8 y códigos HTTP estándar |

En REST, la ruta nombra el recurso (/api/reservas) y el método indica la acción (GET, POST, PUT, DELETE). Los códigos 2xx indican éxito, 4xx un error del cliente y 5xx un error del servidor.

## Preparación del equipo de trabajo (antes de la sesión)

Verifique que la computadora tenga instalado lo siguiente:

| Software | Versión | Para qué se usa |
|---|---|---|
| JDK | 8 o superior (recomendado: 8, 11 o 17) | Compilar y ejecutar el proyecto |
| Eclipse IDE for Java Developers | Una versión reciente | Abrir, compilar y ejecutar el servidor y el cliente |

| Software | Versión | Para qué se usa |
|---|---|---|
| Postman | Cualquier versión reciente | Probar la API sin usar el cliente de consola |

Antes de crear el proyecto, configure Eclipse una sola vez:

1. Cambie a la perspectiva Java: **Window → Perspective → Open Perspective → Java**. Si usa la perspectiva *Java EE*, el menú **File → New** no muestra *Java Project* directamente; en ese caso use **File → New → Project… → Java → Java Project**.
2. Revise que Eclipse reconozca un JDK: **Window → Preferences → Java → Installed JREs**. Si no aparece el JDK, púlselo con **Add… → Standard VM** y la carpeta de instalación del JDK.
3. Ponga UTF-8 como codificación de todo el espacio de trabajo: **Window → Preferences → General → Workspace → Text file encoding → Other: UTF-8 → Apply and Close**.
4. Descomprima **ReservasLabAPI_Semana10.zip** en una carpeta fácil de encontrar (por ejemplo, Documentos). Debe quedar una carpeta **ReservasLabAPI** con la carpeta **src** adentro.

# Parte A · Probar el ejemplo ReservasLabAPI en Eclipse (30 min)

El ejemplo muestra cómo cada RF y RNF se vuelve código. Descargue ReservasLabAPI_Semana10.zip de la plataforma y siga los pasos en orden.

## A.1 Crear el proyecto en Eclipse

1. Vaya a **File → New → Java Project**.
2. En **Project name** escriba `ReservasLabAPI`.
3. Desmarque **Use default location** y, con **Browse…**, seleccione la carpeta **ReservasLabAPI** que descomprimió.
4. En **JRE** elija el JDK instalado y pulse **Next**.
5. En la pestaña **Source**, despliegue **src** y compruebe que aparecen los paquetes `reservas.cliente`, `reservas.datos`, `reservas.modelo`, `reservas.negocio` y `reservas.servidor`. La carpeta de salida (**Default output folder**) debe ser `ReservasLabAPI/bin`. No use el enlace *Add project to build path*.
6. Pulse **Finish**. Si Eclipse pregunta **Create module-info.java**, elija **Don't Create**.
7. Clic derecho en el proyecto → **Properties → Resource → Text file encoding → Other: UTF-8 → Apply and Close**.

## A.2 Revisar que no haya errores

1. Abra **Window → Show View → Problems**. No debe haber errores en rojo.
2. Si aparece **Access restriction: The type HttpServer is not API**, vaya a clic derecho en el proyecto → **Build Path → Configure Build Path → Libraries**, despliegue **JRE System Library**, seleccione **Access rules → Edit… → Add…**, elija **Resolution: Accessible**, escriba el patrón `com/sun/net/httpserver/**` y pulse **OK → Apply and Close**.
3. Si existe un archivo `module-info.java` en **src**, bórrelo.

## A.3 Ejecutar el servidor

1. Abra la vista de consola: **Window → Show View → Console**.
2. En **src → reservas.servidor**, haga clic derecho en **ServidorReservas.java → Run As → Java Application**.
3. En la consola debe aparecer: *Escuchando en http://localhost:8080/api/reservas*. El servidor queda esperando solicitudes; no se abre ninguna ventana.
4. Para comprobarlo, abra en el navegador `http://localhost:8080/api/reservas`. Debe ver la lista de reservas en JSON.

## A.4 Ejecutar el cliente en modo demo

1. Clic derecho en **reservas.cliente → ClienteConsola.java → Run As → Run Configurations…**.
2. Seleccione **Java Application → ClienteConsola** (si no existe, créela con el botón **New launch configuration**).
3. En la pestaña **Arguments → Program arguments** escriba `demo`.
4. En la pestaña **Common → Encoding** elija **UTF-8**.
5. Pulse **Run** y capture la salida.
6. Para cambiar entre la consola del servidor y la del cliente, use el botón **Display Selected Console** (ícono del monitor) en la vista Console. Si prefiere verlas juntas, abra otra con **Open Console → New Console View** y fije cada una con **Pin Console**.

## A.5 Llenar la tabla de resultados

Llene la tabla con lo que observa en la salida del cliente y en la bitácora del servidor.

| Caso de la demo | Código HTTP obtenido | Tiempo (ms) | RF o RNF que verifica |
|---|---|---|---|
| 1. Listar reservas | | | |
| 2. Crear reserva válida | | | |

| Caso de la demo | Código HTTP obtenido | Tiempo (ms) | RF o RNF que verifica |
|---|---|---|---|
| 3. Reserva que choca con #1 | | | |
| 4. Datos incompletos | | | |
| 5. Reserva inexistente | | | |
| 6. Modificar reserva #2 | | | |
| 7. Cancelar reserva #3 | | | |
| 8. Escribir sin API key | | | |
| 9. Prueba de carga (HILOS = 10) | | | |
| 9. Prueba de carga (HILOS = 1) | | | |

## A.6 Probar con el servidor detenido

1. En la consola del servidor, pulse el cuadrado rojo **Terminate**.
2. Ejecute de nuevo el cliente (botón verde **Run** o **Run As → Java Application** sobre ClienteConsola).
3. Anote qué mensaje muestra y qué RNF lo exige.

## A.7 Comparar 10 hilos y 1 hilo

1. Con el servidor funcionando, ejecute el cliente **sin argumentos** (en **Run Configurations → Arguments**, borre `demo`). Aparece el menú en la consola; haga clic dentro de la consola para escribir.
2. Elija la **opción 6** (prueba de carga) y anote los tiempos.
3. Detenga el servidor con **Terminate**. En **ServidorReservas.java** cambie `HILOS = 10` por `HILOS = 1` y guarde con **Ctrl + S**.
4. Ejecute otra vez el servidor y el cliente, repita la opción 6 y compare los tiempos.
5. Al terminar, vuelva a poner `HILOS = 10`.

**Importante:** antes de volver a ejecutar el servidor, detenga siempre el anterior. Si no, aparece el error *Address already in use* porque el puerto 8080 sigue ocupado.

### A.8 Probar con Postman

1. Con el servidor funcionando, abra Postman e importe **ReservasLabAPI.postman_collection.json** (**Import → Upload Files**).
2. Ejecute al menos 3 solicitudes y capture las respuestas.

**Evidencias:** capturas de la salida del cliente, de la bitácora del servidor (consola de Eclipse) y de Postman.

## Parte B · Requisitos de su sistema (40 min)

Trabaje con el escenario cliente-servidor que su equipo definió en la semana 9. Redacte mínimo 6 RF y 5 RNF; al menos uno de cada atributo de la tabla del marco teórico.

### B.1 Ficha detallada

Complete una ficha como esta para los dos RF más importantes:

| Campo | RF-__ |
|---|---|
| Nombre | |
| Actor | |
| Entradas | |
| Proceso / reglas | |
| Salida (éxito y error) | |
| Prioridad (Alta / Media / Baja) | |
| Criterio de aceptación | |

### B.2 Lista de requisitos funcionales

| ID | Descripción (“El sistema permitirá…”) | Actor | Prioridad |
|---|---|---|---|
| RF-01 | | | |
| RF-02 | | | |
| RF-03 | | | |
| RF-04 | | | |
| RF-05 | | | |

| ID | Descripción (“El sistema permitirá…”) | Actor | Prioridad |
|---|---|---|---|
| RF-06 | | | |

### B.3 Requisitos no funcionales

Cada RNF debe tener métrica y valor objetivo; “rápido” o “seguro” no se aceptan.

| ID | Atributo | Requisito medible | Cómo se verificará | Decisión de diseño que provoca |
|---|---|---|---|---|
| RNF-01 | Seguridad | | | |
| RNF-02 | Rendimiento | | | |
| RNF-03 | Disponibilidad | | | |
| RNF-04 | Concurrencia | | | |
| RNF-05 | Interoperabilidad | | | |

## Parte C · Selección del framework (15 min)

Califique cada opción de 1 (malo) a 5 (excelente) según su equipo; el peso refleja qué tanto importa cada criterio. Puntaje = suma de (calificación × peso). Puede reemplazar una columna por otro framework.

| Criterio | Peso | JDK HttpServer (Java 8) | Spring Boot 2.7 | Express (Node.js) | Flask (Python) |
|---|---|---|---|---|---|
| Dominio del lenguaje en el equipo | 3 | | | | |
| Facilidad para cumplir los RNF | 3 | | | | |
| Curva de aprendizaje | 2 | | | | |
| Documentación y comunidad | 1 | | | | |
| Requisitos de la computadora del laboratorio | 1 | | | | |
| **Puntaje total** | | | | | |

Redacte la justificación en un párrafo de 4 a 6 líneas: framework elegido, criterio decisivo y qué RNF facilita. Si eligen Spring Boot con Java 8, usen la versión 2.7; la 3.x requiere Java 17. En Eclipse, un proyecto Spring Boot se crea como proyecto Maven (**File → New → Maven Project**) o con el complemento Spring Tools.

## Parte D · Documentar los endpoints (15 min)

Cada RF que el cliente invoca se convierte en un endpoint. Use el archivo **CONTRATO_API.md** del ejemplo como modelo; en Eclipse puede abrirlo con doble clic desde el **Package Explorer**.

| RF | Método | Ruta | Cuerpo de la solicitud | Respuesta exitosa | Errores posibles |
|---|---|---|---|---|---|
| RF-01 | | | | | |
| RF-02 | | | | | |
| RF-03 | | | | | |
| RF-04 | | | | | |
| RF-05 | | | | | |

Incluya además un ejemplo completo de solicitud y respuesta en JSON (éxito y error) para el endpoint de creación, como este del ejemplo:

```
POST /api/reservas
X-API-KEY: unachi-prog5

{"laboratorio":"LAB-3","solicitante":"Ana Pérez","fecha":"2026-10-
24","horaInicio":"08:00","horaFin":"10:00"}

201 Created → {"id":4,"laboratorio":"LAB-3","estado":"ACTIVA", ...}
409 Conflict → {"error":409,"mensaje":"Choque de horario con la reserva #1
(07:00-08:35)"}
```

## Preguntas de reflexión

Responda en 3 a 5 líneas cada una.

1. ¿Qué requisitos no funcionales son especialmente críticos en una arquitectura cliente-servidor y por qué? (desafío cognitivo)
2. En el ejemplo, ¿por qué RF-06 y RF-07 no tienen endpoint propio? ¿En qué capa viven?
3. ¿Qué diferencia observó en la prueba de carga con 1 hilo y con 10 hilos? ¿Qué RNF se ve afectado?
4. ¿Por qué el cliente debe tener timeouts aunque el servidor funcione bien la mayor parte del tiempo?
5. ¿Qué cambiaría en sus RNF si el sistema tuviera también un cliente móvil con conexión lenta?

## Entregables y formato del informe

Un solo PDF por equipo, nombrado **Lab10_Equipo#.pdf**, subido por el coordinador del equipo, con la estructura del informe de laboratorio del curso:

1. Portada (integrantes y número de equipo)
2. Objetivo del laboratorio
3. Marco teórico breve (media página, con sus palabras)
4. Procedimiento realizado: Partes A a D
5. Capturas de pantalla y evidencias: salida del cliente, bitácora del servidor (consola de Eclipse), Postman
6. Resultados obtenidos: tablas de RF, RNF, matriz de decisión y endpoints
7. Conclusiones, incluidas las respuestas a las preguntas de reflexión

Este SRS es la base de la semana 11 (diagrama de arquitectura y contrato de API) y de la implementación de la semana 12.

# Rúbrica de evaluación (100 puntos)

| Criterio | Excelente | Bueno | En desarrollo | Puntos |
|---|---|---|---|---|
| Parte A · Prueba del ejemplo y evidencias | Tabla completa, capturas claras, comparación 1 vs 10 hilos explicada | Tabla completa con capturas parciales | Tabla incompleta o sin evidencias | 15 |
| Parte B · Requisitos funcionales | ⩾ 6 RF claros, con actor, prioridad y 2 fichas completas | 6 RF con algunas ambigüedades | Menos de 6 RF o sin fichas | 25 |
| Parte B · Requisitos no funcionales | ⩾ 5 RNF medibles, con verificación y decisión de diseño | RNF con métrica pero sin verificación | RNF vagos (“rápido”, “seguro”) | 25 |
| Parte C · Selección del framework | Matriz completa y justificación con criterio decisivo | Matriz completa, justificación débil | Sin matriz o sin justificación | 10 |
| Parte D · Endpoints | Tabla completa, rutas REST correctas, ejemplo JSON de éxito y error | Tabla completa sin ejemplo JSON | Rutas no REST o tabla incompleta | 15 |
| Reflexión y formato del informe | Respuestas fundamentadas, informe con las 7 secciones | Respuestas breves, faltan 1–2 secciones | Sin reflexión o informe desordenado | 10 |
| **Total** | | | | **100** |

# Anexo · Solución de problemas frecuentes en Eclipse

| Problema | Causa | Solución |
|---|---|---|
| El menú File → New no muestra *Java Project* | Está abierta la perspectiva Java EE | **File → New → Project… → Java → Java Project**, o cambie a la perspectiva Java |
| *Access restriction: The type HttpServer is not API* | Eclipse restringe el paquete `com.sun.net.httpserver` | Agregue la regla de acceso `com/sun/net/httpserver/**` como **Accessible** (paso A.2) |

| Problema | Causa | Solución |
|---|---|---|
| *The package com.sun.net.httpserver is not accessible* | Se creó `module-info.java` | Borre `module-info.java` de **src** |
| Run As → Java Application no hace nada visible | La vista Console está cerrada | **Window → Show View → Console**; el servidor no abre ventanas, solo escribe en la consola |
| No aparece *Java Application* en Run As | Se hizo clic derecho en una carpeta o paquete | Haga clic derecho sobre el archivo `.java` que tiene el método `main` |
| *Address already in use* o *BindException* | Hay otro servidor usando el puerto 8080 | Pulse **Terminate** en la consola del servidor anterior (o use **Remove All Terminated Launches**) y ejecute de nuevo |
| El cliente muestra *No se pudo conectar* | El servidor no está en ejecución | Ejecute primero **ServidorReservas.java** |
| Las tildes y la ñ se ven mal | Codificación distinta de UTF-8 | UTF-8 en **Properties → Resource** y en **Run Configurations → Common → Encoding** |
| No se puede escribir en el menú del cliente | El cursor no está en la consola del cliente | Seleccione la consola del cliente y haga clic dentro antes de escribir |